

Experiment - 1

1. WAP to declare a class student having data members or name, roll no., accept and display data for one student.

```
#include <stdio.h>
```

```
using namespace std;
```

```
class student{
```

```
    String name;
```

```
    int roll-no;
```

```
    public
```

```
    void accept ();
```

```
    void display ();
```

```
};
```

```
void accept ();
```

```
void student::accept () {
```

```
    cout << "Enter name and roll no. - ";
```

```
    in >> name >> roll-no;
```

```
    public
```

```
    void accept ();
```

```
}
```

```
cout << "\n student name: " << name;
```

```
cout << "\n student roll no: " << roll-no;
```

```
}
```

```
int main () {
```

```
    student <I;
```

```
    SI accept ();
```

```
    SI display ();
```

```
    return 0;
```

```
}
```

Output:

Enter name and roll no. - abc

23

Student name: abc

Student roll no: 23

3. WAP to declare a class time having data members as H, m, s. Accept data for one object & display data of total time in seconds.

```
#include <iostream>
```

```
using namespace std;
```

```
class time {
```

```
int H, m, s;
```

```
public:
```

```
void accept();
```

```
void display seconds();
```

```
};
```

```
void time::accept() {
```

```
cout << "\n Enter hours:";
```

```
cin >> H;
```

```
cout << "Enter minutes:";
```

```
in >> m;
```

```
cout << "Enter seconds:";
```

```
in >> s;
```

```
}
```

```
void time::display seconds() {
```

```
int total = (H * 3600) + (m * 60) + s;
```

```
cout << "\n Total time in seconds:" <<
```

```
total << endl;
```

```
}
```

```
int main() {
```

```
time t;
```

```
cout << "Enter time details:";
```

```
t.accept();
```

```
t.display seconds();
```

```
return 0;
```

```
}
```

Output:

Enter the details:

Enter hours: 2

Enter minutes: 10

Enter seconds: 45

Total time in seconds: 7845

2. WAP to declare a class book. having data members as id, name, price. Accept data for 2 books and display data of book having greater price.

```
#include <iostream>
```

```
using namespace std;
```

```
class book {
```

```
int id;
```

```
string name;
```

```
float price;
```

```
public:
```

```
void accept();
```

```
void display();
```

```
float get price();
```

```
}
```

```
void book::accept() {
```

```
cout << "\n Enter book Id:";
```



```

cin >> id;
cout << "Enter book name: ";
in >> name;
cout << "Enter price: ";
cin >> price;
cin >> price;
}
void book::display() {
    cout << "In Book ID: " << id << endl;
    cout << "Price: " << price << endl;
}
float book::get price() {
    return price;
}
int main() {
    book b1, b2;
    cout << "Enter details for book 1: ";
    b1.accept();
    cout << "Enter details for book 2: ";
    b2.accept();
    cout << "In Book with greater price is: ";
    if (b1.get price() > b2.get price()) {
        b1.display();
    } else {
        b2.display();
    }
    return 0;
}

```

Output :

```

Enter details for book 1:
Enter book name: Twilight
Enter price: 520
Enter details for book 2:
Enter book ID: 12640
Enter book name: Divergent
Enter book price: 460
Book with greater price is :
Book ID: 12640
Book name: Divergent
Price: 460.

```

29/7/25

Experiment-2

1. WAP to declare a class 'city' having data members as a name and population. Accept this data for 5 cities and display name of city having highest population.

```
#include <iostream>
using namespace std;
class city {
public:
    string name;
    int population;
};
int main() {
    city cities[5];
    string maxcity;
    int maxpop = 0;
    for (int i = 0; i < 5; i++) {
        cout << "Enter name of city " << i+1 << ": ";
        cin >> cities[i].name;
        cout << "Enter population: ";
        cin >> cities[i].population;
        if (cities[i].population > maxpop) {
            maxpop = cities[i].population;
            maxcity = cities[i].name;
        }
    }
    cout << "City with highest population: "
    << maxcity << endl;
    return 0;
}
```

Output-

Enter name of city 1: Pune

Enter population: 30000

Enter name of city 2: Nagpur

population: 5000

3: Mumbai

population: 20000

4: Hyderabad

population: 40000

5: Kolkata

population: 80000

City with highest population: Kolkata

2. WAP to declare a class "Account" having data members as Account no. and balance. Accept this data for 10 accounts and give interest of 10% where balance is equal or greater than 5000 and display them:

```
#include <iostream>
using namespace std;
class Account {
public:
    int accNo;
    float balance;
};

int main() {
    Account acc[10];
    for (int i=0; i<10; i++) {
        cout << "\nEnter details for Account "<i+1<
        "<< endl;
        cout << "Account No: ";
        cin >> acc[i].accNo;
        cout << "Balance: ";
        cin >> acc[i].balance;

        if (acc[i].balance >= 5000) {
            acc[i].balance = acc[i].balance * 1.10;
        }
    }
    cout << "\n - - - Accounts with balance >= 5000  
after 10% interest - - - \n";
```

```
for (int i=0; i<10; i++) {
    if (acc[i].balance >= 5000) {
        cout << "Account "<i+1<< " Account No: "
        << acc[i].accNo
        << " Update Balance: "<< acc[i].balance << endl;
    }
}
return 0;
}
```

Output:

Account No: 76645

Balance: 5555

Enter details for Accounts 5

Account No: 4356

Balance: 6666

Enter details for Account 6

Account No: 88775

Balance: 7777

Enter details for Account 7.

Account No: 3635

Balance: 8888

Enter details for Account 8

Account No: 454543

Balance: 9999

Enter details for Account 9

Account No: 1212

Balance: 1213

Enter details for Account 10

Account No: 65978

Balance: 1424

Accounts with Balance ≥ 5000 (after 10% interest)

Account 4 - Account No: 76045, updated balance: 6110.5

Account 5 - Account No: 4356, updated balance: 7332.6

Account 6 - Account No: 88775, - 11%: 8554.7

Account 7 - Account No: 3635, - 11%: 9776.8

Account 8 - Account No: 454543, updated - 11%: 109989

3. WAP to declare a class "staff" having data members as name and post. Accept this data members for 5 staff and display of staff who are "HOD".

```
→ #include <iostream>
#include <string>
using namespace std;
class staff {
public:
    char name[50];
    char post[20];
};

int main() {
    staff s[5];
    for (int i = 0; i < 5; i++) {
        cout << "Enter name of staff " << i + 1 << ": ";
        cin >> s[i].name;
        cout << "Enter post: ";
        cin >> s[i].post;
    }
    cout << "staff with post 'HOD': \n";
    for (int i = 0; i < 5; i++) {
        if (strcmp(s[i].post, "HOD") == 0) {
            cout << s[i].name << endl;
        }
    }
    return 0;
}
```


Output

Enter name of staff 1: Shruti

Enter post: HOD

-11-

2: Ridhima

post: Faculty

-11-

3: Gayatri

post: Faculty

-11-

4: Sneha

post: Faculty

-11-

5: Vishal

post: Faculty

Staff with post 'HOD':

Shruti

Pr
29/7/25

Experiment-3

- a) WAP to declare class 'book' containing data members as book_title, author, name and price. Accept and display the information for one object using pointer to that class.

#include <stdio.h>

using namespace std;

class Book

{

string t, a;

float p;

public:

void accept()

{

cout << "Enter book title: ";

getline (cin, t);

cout << "Enter author name: ";

getline (cin, a);

cout << "Enter price: ";

cin >> p;

}

void disp()

{

cout << "Book title: " << t;

cout << "Book author: " << a;

cout << "Price: " << p;

};

int main()

{

Book b1;

Book *ptr = &b1;

ptr -> accept();

ptr → dis ()
return 0;
}

Output:

Enter BOOK Title: ABC
Enter author name: XYZ
Enter Price: 500

BOOK Title: ABC
BOOK Author: XYZ
Price: 500

- b) WAP to declare a class 'student' having data members roll-no and percentage. Using 'this' pointer involve member functions to accept and display the data for one object of the class.

```
→ #include <iostream>
using namespace std;

class student
{
    int r;
    float p;
    public:
    void accept (int r, float p)
    {
        this → r = r;
        this → p = p;
    }
};
```

void dis ()

```
{
    cout << "Roll no:" << r << endl;
    cout << "Percentage : " << p << "%\n" << endl;
}
```

};

int main ()

```
{
    student s;
    int ro;
    int pe;
    cout << "Enter roll no: ";
    cin >> ro;
    cout << "Enter percentage: ";
    cin >> pe;
    s.accept (ro, pe);
    s.dis ();
    return 0;
}
```

output:

Enter roll no: 33
Enter percentage: 92.2
Roll no: 33
Percentage: 92.2 %

Ph
4/11

Experiment - 4

1. Create 2 classes, class A and B, each with a private integer. Write a friend function sum() that can access private data from both classes and return sum.

```
→ #include <iostream>
using namespace std;
```

```
class B;
```

```
class A {
```

```
private:
    int x;
```

```
public:
```

```
    A(int val) {
        x = val;
    }
```

```
    friend int sum(A, B);
```

```
};
```

```
class B {
```

```
private:
```

```
    int y;
```

```
public:
```

```
    B(int val) {
```

```
        y = val;
    }
```

```
    friend int sum(A, B);
```

```
};
```

```
int sum(Aa, Bb) {
```

```
    return a + b;
```

```
}
```

```
int main() {
```

```
    A obj1(5);
```

```
    B obj2(7);
```

```
    cout << "sum: " << sum(obj1, obj2) << endl;
```

```
    return 0;
```

```
}
```

Output: sum = 12

2. WAP With class no. that contains a private integer. Use a friend function swap No. (Number & number) to swap the private values of two no. object.

```
→ #include <iostream>
using namespace std;
```

```
class Number {
private:
    int value;
public:
    Number (int val) {
        value = val;
    }
    void show () {
```

```
        cout << "value" << value << endl;
```

```
    }
```

```
    friend void swap numbers (Number & n1, Number & n2);
```

```
};
```

```
void swap numbers (Number & n1, Number & n2) {
```

```
    int temp = n1.value;
```

```
    n1.value = n2.value;
```

```
}
```

```
int main () {
```

```
    Number a (10);
```

```
    Number b (20);
```

```
    cout << "Before swap" << endl;
```

```
    a.show ();
```

```
    b.show ();
```

```
    swap numbers (a, b);
```

```
    cout << "After swap:" << endl;
```

```
    a.show ();
```

```
    b.show ();
```

```
    return 0;
```

```
}
```

output:

Before swap:-

value: 10

value: 20

After swap:-

value: 20

value: 10

3. Define 2 classes Box and Cube, each having a private vol^m. Write a friend fn. find greater (Box, cube) then determines which obj has a larger vol^m.

```
#include <iostream>
```

```
using namespace std;
```

```
class cube;
```

```
class Box {
```

```
private:
```

```
int volume;
```

```
public:
```

```
Box (int v) {
```

```
volume = v;
```

```
}
```

```
friend void find greater (Box, cube);
```

```
};
```

```
class cube {
```

```
private:
```

```
int volume;
```

```
public:
```

```
cube (int v) {
```

```
volume = v;
```

```
}
```

```
friend void find greater (Box, cube);
```

```
};
```

```
void find greater (Box b, cube c) {  
    if (b.volume > c.volume) {  
        cout << "Box has greater volume:" <<  
    } else if (c.volume > b.volume) { b.volume << endl;  
        cout << "cube has greater volume:" << c.volume << endl;  
    }  
    else {  
        cout << "Both have equal volume:" << b.volume  
        << endl;  
    }  
}
```

```
int main () {
```

```
Box box1 (100);
```

```
cube box2 (80);
```

```
find greater (box1, cube1);  
return 0;
```

```
}
```

Output: Box has greater volume:100

→ create a class complex with real and imaginary parts as private members. Use a friend function to add two complex no and return the result as a new complex object.

```
→ #include <iostream>
    using namespace std;

    class complex {
    private:
        int real;
        int img;

    public:
        complex (int r=0, int i=0) {
            real = r;
            image = i;
        }
        void show() {
            cout << real << " + " << image << "i" << endl;
        }
        friend complex add (complex, complex);
    };

    complex add (complex c1, complex c2) {
        complex result;
        result.real = c1.real + c2.real;
        result.image = c1.image + c2.image;
        return result;
    }
```

```
int main() {
    complex a(2, 3);
    complex b(4, 5);
```

```
    complex c = add(a, b);
    cout << "Sum of complex no. : ";
    c.show();
    return 0;
```

```
}

Output:
Sum of complex numbers : 6 + 8i
```


5. Create a class stu. with private data members: name and 3 subject marks. Write a friend function cal. Average(stu.) that cal. and display the avg. marks.

```
#include <iostream>
using namespace std;
class student {
private:
    string name;
    int mark 1, mark 2, mark 3;
public:
    student (string n, int m1, int m2, int m3) {
        name = n;
        mark1 = m1;
        mark2 = m2;
        mark3 = m3;
    }
};
```

Friend void calculate Average(student)

```
{
void calculate average (students) {
    float avg = (s.mark1 + s.mark2 + s.mark3) / 3;
    cout << "Student name: " << s.name << endl;
    cout << "Average marks: " << avg << endl;
}
int main() {
    student s ("Ani", 85, 90, 98);
    calculate Average (s);
    return 0;
}
```

Output:

Student name- Ani
Average marks- 90

6. Create three classes: Alpha, Beta and Gamma, each with a private data member write a single friend fn that can access all three and print their sum.

```
#include <iostream>
using namespace std;
```

```
class Beta;
class Gamma;
```

```
class Alpha {
private:
```

public:

```
Alpha (int val) {
    a = val;
```

```
friend void total sum (Alpha, Beta, Gamma);
};
```

```
class Beta {
private:
    int b;
```

public:

```
Beta (int val)
    b = val;
```

```
};
```

```
friend void total sum (Alpha, Beta, Gamma)
};
```

```
class Gamma {
private:
    int;
```

```
public:
    Gamma (int val) {
        c = val;
    }
```

```
friend void total sum (Alpha, Beta, Gamma)
};
```

```
void total sum (Alpha x, Beta y, Gamma z) {
    int sum = x.a + y.b + z.c;
    cout << "sum of all values : " << sum << endl;
}
```

```
int main () {
    Alpha obj 1 (10);
    Beta obj 2 (20);
    Gamma obj 3 (30);
```

```
total sum (obj 1, obj 2, obj 3);
return 0;
}
```

output:
sum of all values : 60

7. Create a class point with private members x and y. Write a friend function that cal. and returns the distance betn two points object.

```
#include <iostream>
#include <math>
using namespace std;
```

```
class point {
private:
    int x, y;
```

```
public:
    point (int x val, int y val) {
        x = x val;
        y = y val;
    }
```

```
friend double calculate distance (point, point);
};
```

```
double calculate distance (point p1, point p2) {
    int dx = p2.x - p1.x;
    int dy = p2.y - p1.y;
    return sqrt (dx * dx + dy * dy);
}
```

```
int main () {
    point p1 (3, 4);
    point p2 (7, 1);
```

```
double distance = calculate distance (p1, p2);
```


cout << "Distance between points: " << distance << endl;

return 0;
}

Output -
Distance between points

8. Create two classes: Bank Acc and Audit. Bank Account holds private balance information. Write a friend function in Audit that accesses and prints balance info for auditing.

```
#include <iostream>
using namespace std;
```

```
class Audit;
class Bank account {
private:
    int balance;
```

```
public:
    Bank Account(int b){
        balance = b;
```

```
    }
    friend void audit Balance (Bank Account,
                                Audit);
```

```
class Audit {
public:
    void start Audit (Bank account acc){
        audit Balance (acc, *this);
```

```
    }
    friend void audit Balance (Bank acc, Audit);
};
```

```
void audit Balance (Bank Account acc, Audit){
    cout << "Balance : rs." << acc. Balance << endl;
```

```
};

int main(){
    Bank Account my acc (50000);
    Audit auditor;
```

auditor.start Audit (Any A());

return 0;

}

Output:

Balance: rs. 50000

- g. WAP to swap 2 no. from same class using object as an argument. Write swap fⁿ as member fⁿ.

```
#include <iostream>
```

```
using namespace std;
```

```
class Number{
```

```
{ int num; (n int) ;
```

```
Public:
```

```
    ~Number (int n) { num = n; }
```

```
    ~void swap (Number &n) {
```

```
        int t = num;
```

```
        num = n.num;
```

```
        n.num = t;
```

```
    }
```

```
    void show () {
```

```
        cout << num << " "; }
```

```
};
```

```
int main () {
```

```
    Number a (10), b (20);
```

```
    a.show (); b.show ();
```

```
    cout << endl;
```

```
    a.swap (b);
```

```
    a.show (); b.show ();
```

```
    b.show ();
```

Output:

10 20

20 10

10. WAP to swap 2 no. from same class using concept of Friend Function.

```
#include <iostream>
```

```
using namespace std;
```

```
class Number {
```

```
    int num;
```

```
public:
```

```
    Number(int n) { num = n; }
```

```
    friend void swap numbers (Number &n,
```

```
                                Number &y);
```

```
    void show () {
```

```
        cout << num << " "; }
```

```
};
```

```
void swap numbers (Number &x, Number &y)
```

```
{
```

```
    int temp = x.num;
```

```
    x.num = y.num;
```

```
    y.num = temp;
```

```
}
```

```
int main () {
```

```
    Number a(10), b(20);
```

```
    a.show (); b.show (); cout << endl;
```

```
    swap numbers (a, b);
```

```
    a.show ();
```

```
    b.show ();
```

```
}
```

11. WAP to swap 2 no. from diff. class using Friend Function.

```
#include <iostream>
```

```
using namespace std;
```

```
class B;
```

```
class A {
```

```
    int numA;
```

```
public:
```

```
    A(int n) { numA = n; }
```

```
    friend void swap numbers (A &n, B &y);
```

```
    void show () {
```

```
        cout << numA << " "; }
```

```
};
```

```
class B {
```

```
    int numB;
```

```
public:
```

```
    B(int n) { numB = n; }
```

```
    friend void swap numbers (A &n, B &y);
```

```
    void show () {
```

```
        cout << numB << " "; }
```

```
};
```

```
void swap numbers (A &n, B &y) {
```

```
    int temp = n.numA
```

```
    n.numA = y.numB
```

```
    y.numB = temp;
```

```
}
```

```
int main () {
```

```
    A obj 1(10);
```

```
    B obj 2(20);
```

```
obj1.show();
obj2.show();
cout<<endl;
swap numbers (obj1, obj2);
obj1.show(); obj2.show();
}
```

Output:

```
10 20
20 10
```

12. WAP to create 2 classes Result 1 and Result 2 which stores the marks of the stu.

→ #include <iostream>

using namespace std;

class Result 1;

class Result 2 {

Float marks;

public;

void read() {

cout<<"Enter marks for result 1:";

cin>>marks;

}

friend void average (result 1, result 2);

};

class Result 2 {

Float marks;

public:

void read() {

cout<<"Enter mark for Result 2:";

cin>>marks;

}

friend void average (Result 1, Result 2);

}

void average (Result 1, Result 2) {

Float avg = (r1.marks + r2.marks) / 2

cout<<"Average marks="<<avg<<endl;

}

int main() {


```
Result 1 Obj 1;
Result 2 Obj 2;

Obj 1.read();
Obj 2.read();

Average (Obj 1, Obj 2);
return 0;
}
```

output:

Enter marks for Result 1: 100
Enter marks for Result 2: 99
Average marks: 99.5

Experiment-5

- Q) WAP to find the sum of nos. between 1 to n using a constructor where the value of n will be passed to the constructor.

→ 1) Using Default constructor.

```
#include <iostream>
using namespace std;
class sum
{
    int n, s;
public:
    sum()
    {
        n = 10;
    }
    void calculate()
    {
        for (int i = 1; i <= n; i++)
        {
            s = s + i;
        }
    }
    void display()
    {
        cout << "The sum from 1 to n is: " << s;
    }
};

int main()
{
    sum s1;
    s1.calculate();
    s1.display();
    return 0;
}
```

2) By Using Parameterized constructor:

```
#include <iostream>
using namespace std;
class sum
{
    int n, s = 0;
    public:
        sum(int num)
        {
            n = num;
        }
        void calculate()
        {
            for (int i = 1; i <= n; i++)
            {
                s = s + i;
            }
        }
        void display()
        {
            cout << "The sum from 1 to n is: " << s;
        }
};

int main()
{
    sum s1(10);
    s1.calculate();
    s1.display();
    return 0;
}
```

3) By using copy constructor.

```
#include <iostream>
using namespace std;
class sum
{
    int n, s = 0;
    public:
        sum(int num)
        {
            n = num;
        }
        sum(sum & s1)
        {
            n = s1.n;
            for (int i = 1; i <= n; i++)
            {
                s = s + i;
            }
        }
        cout << "The sum from 1 to n is: " << s;
    };
};

int main()
{
    sum s1(10);
    sum s2(s1);
    return 0;
}
```

#output: The sum from 1 to n is: 55

b) Write a program to declare a class "Student" having data members as name and percent. Write a constructor to initialize these data members. Accept and display data for 1 student.

```
#include <iostream>
using namespace std;

class Student {
public:
    String name;
    Float percentage;

    Student() {
        name = "Ridhima";
        percentage = 100;
    }

    Student (String n, Float p) {
        name = n;
        percentage = p;
    }

    Student (Student & obj) {
        name = obj.name;
        percentage = obj.percentage;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Percentage: " << percentage << endl;
    }
};
```

```
int main() {
    String n;
    Float p;
    cout << "Enter student name: ";
    getline (cin, n);

    cout << "Enter student percentage: ";
    cin >> p;

    Student s1(n, p);
    s1.display();
    Student s2(s1);

    cout << "In Data of copied Student: " <<
        endl;
    s2.display();

    Student s3;
    cout << "In Data of default student: "
        << endl;
    s3.display();

    return 0;
}
```

classmate
Date _____
Page _____

5) Define a class "college" members variables as roll-no, name, course. WAP using constructor with default value as "computer Engineer" for course. Accept this data for 2 objects of class and display the data.

```
→ #include <iostream>
using namespace std;

class college
{
    string name, course;
    int roll;
public:
    college (string n, int r, string c = "computer engineering")
    {
        name = n;
        course = c;
        roll = r;
    }
    college (string n, string c = "computer engineering", int r = 123)
    {
        name = n;
        course = c;
        roll = r;
    }
    void display ()
    {
```

```
cout << "The name of the student is: " << name << endl;
cout << "The course of the student is: " << course << endl;
cout << "The roll no. of student: " << roll << endl;
}
};

int main ()
{
    college c1 ("Ridhima", 43);
    c1.display ();
    college c2 ("Gayatri");
    c2.display ();
    return 0;
}
```

Output:

```
The name of student is: Ridhima
The course of the student is: computer engineering
The roll no. of student: 43
The name of the student is: gayatri
The course of the student is: computer eng.
The roll no. of student: 39
```


d) write a c++ program to demonstrate constructor overloading.

```
→ #include <iostream>
using namespace std;
class rectangle
{
    int l, b;
public:
    rectangle()
    {
        l=2;
        b=5;
    }
    rectangle(int x)
    {
        l=x;
        b=x;
    }
    rectangle(int x, int y)
    {
        l=x;
        b=y;
    }
    rectangle(rectangle &r3)
    {
        l=r3.l;
        b=r3.b;
    }
}
```

```
void calculate()
{
}
void calculate()
{
    cout << "The area is: " << l * b << endl;
}
int main()
{
    rectangle r1;
    r1.calculate();
    rectangle r2(3);
    r2.calculate();
    rectangle r3(6, 6);
    r3.calculate();
    rectangle r4(r3);
    r4.calculate();
    return 0;
}
```

output:
The area is: 10
The area is: 9
The area is: 36
The area is: 36

Experiment - 6

1. Single Inheritance

Create a base class called person with attributes name and age. Derive a class student from person that adds an attribute roll number, name and class.

```
#include <iostream>
using namespace std;
class Department {
protected:
    string dname;
};
class student: protected Department {
protected:
    string sname;
    int roll;
};
class student: protected Department {
protected:
    string sname;
    int m1, m2, percentage;
public:
    void accept() {
        cout << "Enter department : " << endl;
        cin >> dname;
        cout << "Enter name : " << endl;
        cin >> sname;
        cout << "Enter marks 1 : " << endl;
        cin >> m1;
        cout << "Enter marks 2 : " << endl;
        cin >> m2;
    }
};
```

```
void calculate() {
    int per = (m1 + m2) / 2;
    cout << "Department : " << dname << endl;
    cout << "Name : " << sname << endl;
    cout << "Percentage : " << per << endl;
}
};
int main() {
    marks m;
    m.accept();
    m.calculate();
    return 0;
}
```

Output:

Enter department:

Ridhima

Enter marks 1:

88

Enter marks 2:

91

Department: CSE

Name: Ridhima

Per : 89

Q2) WAP to Implement multiple inheritance.

Assume suitable data.

```
#include <iostream>
```

```
using namespace std;
```

```
class department {
```

```
protected:
```

```
string dname;
```

```
};
```

```
class student {
```

```
protected:
```

```
string sname;
```

```
int roll;
```

```
};
```

```
class marks : protected department, protected student {
```

```
int m1, m2, percentage;
```

```
public:
```

```
void accept () {
```

```
cout << "Enter dept : " << endl;
```

```
cin >> dname;
```

```
cout << "Enter name : " << endl;
```

```
cin >> sname;
```

```
cout << "Enter marks 1 : " << endl;
```

```
cin >> m1;
```

```
cout << "Enter marks 2 : " << endl;
```

```
cin >> m2;
```

```
}
```

```
void calculate () {
```

```
int per = (m1 + m2) / 2
```

```
cout << "Dept : " << dname << endl;
```

```
cout << "Name : " << sname << endl;
```

```
cout << "Percentage : " << endl;
```

```
}
```

```
};
```

```
int main () {
```

```
marks m;
```

```
m.accept ();
```

```
m.calculate ();
```

```
}
```

Q3) WAP to implement hierarchical inheritance.

```
#include <iostream>
```

```
using namespace std;
```

```
class Person {
```

```
protected:
```

```
string name;
```

```
int age;
```

```
public:
```

```
void accept () {
```

```
cout << "Enter name : " << endl;
```

```
cin >> name;
```

```
cout << "Enter age : " << endl;
```

```
cin >> age;
```

```
}
```

```
};
```

classmate
Date _____
Page _____

```

class Student : public Person {
    int roll;
    float per;
public:
    void acceptstud() {
        cout << "Enter roll number: << endl;
        cin >> roll;
        cout << "Enter percentage: " << endl;
        cin >> per;
    }
    void display stud() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "Rollno: " << Roll << endl;
        cout << "Percentage: " << Per << endl;
    }
};

```

```

class Staff : public person {
    int emp_id;
    string subject;
public:
    void acceptstaff() {
        cout << "Enter employee ID: " << endl;
        cin >> emp_id;
        cout << "Enter subject: " << endl;
        cin >> subject;
    }
    void displaystaff() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "Empid: " << emp_id << endl;
    }
};

```

classmate
Date _____
Page _____

```

        cout << "subject taught: " << sub << endl;
    }
};

int main() {
    Student s;
    Staff t;

    s.acceptPer();
    s.acceptstud();
    t.acceptPer();
    t.acceptstaff();

    s.displaystud();
    t.displaystaff();

    return 0;
}

```

e) WAP to demonstrate virtual base class. Assume suitable data with figures.

```

#include <iostream>
using namespace std;

```

```

class CollegeStudent {
protected:
    int student-id;
    string code;
public:
    void accept() {
        cout << "Enter student ID: ";
        cin >> student-id;
    }
};

```


classmate
Date _____
Page _____

```

cout << "Enter college code:";
cin >> code;
}
void display() {
    cout << "Student ID:" << student_id << endl;
    cout << "College code:" << code << endl;
}
};

class Test : virtual public CollegeStudent {
protected:
    float percentage;
public:
    void accept() {
        CollegeStudent::accept();
        cout << "Enter Test percentage:";
        cin >> percentage;
    }
    void display() {
        CollegeStudent::display();
        cout << "Test percentage:" << percentage
        << "%." << endl;
    }
};

class Sports : virtual public CollegeStudent {
protected:
    int grade;
public:
    void accept() {
        cout << "Enter sports grade:";
        cin >> grade;
    }
}

```

classmate
Date _____
Page _____

```

void display() {
    cout << "Sports grade:" << grade << endl;
}
};

class Result : public Test, public Sports {
    float tot_marks;
public:
    void accept() {
        CollegeStudent::accept();
        cout << "Enter Test percentage:";
        cin >> percentage;
        cout << "Enter sports Grade:";
        cin >> grade;
        cout << "Enter total marks:";
        cin >> tot_marks;
    }
    void display() {
        CollegeStudent::display();
        cout << "Test percentage" << percentage
        << "%." << endl;
        cout << "Sports grade:" << grade << endl;
        cout << "Total marks:" << tot_marks << endl;
    }
};

int main() {
    Result r;
    cout << "Enter stu. details" << endl;
    r.accept();
    r.display();
    return 0;
}

```

4/11

Experiment - 7

Ques) WAP using function overloading to calculate the area of a laboratory (rectangular) and area of Classroom (square).

```
#include <iostream>
using namespace std;
```

```
class Area {
public:
    float calculate(float length, float breadth) {
        return length * breadth;
    }
}
```

```
    float calculate(float side) {
        return side * side;
    }
};
```

```
int main() {
    Area a;
    cout << "Area of Laboratory:" << a.calculate(10, 6) << endl;
    cout << "Area of Square:" << a.calculate(5) << endl;
}
```

Q2) WAP using function overloading to calculate the sum of 5 float values and sum of 10 integer values.

```
#include <iostream>
using namespace std;
```

```
class sum {
public:
    int total(int a[], int n) {
        int s = 0;
        for (int i = 0; i < n; i++)
            s += a[i];
        return s;
    }
}
```

```
float total(float a[], int n) {
    float s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];
    return s;
}
```

```
int main() {
    sum s;
    int marks[10] = {45, 56, 67, 78, 89, 90, 76, 88, 92, 85};
    float grades[5] = {9.2, 8.7, 9.5, 8.9, 9.0};
}
```

```
cout << "sum of 10 stu marks:" << s.total(marks, 10) << endl;
```

```
cout << "sum of 5 stu marks points:" << s.total(grades, 5) << endl;
```


- 3) WAP to demonstrate the compile Time and implement Unary-operator when used with the object so that the numeric data member of the class is negated.

```
#include <iostream>
using namespace std;
```

```
class teacher {
    int experience;
public:
    Teacher(int e) {
        experience = e;
    }
    void display() {
        cout << "Experience:" << experience << "years"
        << endl;
    }
    void operator-() {
        experience = -experience;
    }
};

int main() {
    Teacher t1(10);
    t1.display();
    -t1;
    cout << "After negation:";
    t1.display();
    return 0;
}
```

- 34) WAP to implement the Unary++ operator when used the object so that the numeric data member of the class is incremented.

```
#include <iostream>
using namespace std;
```

```
class student {
    int count;
public:
    student(int c = 0) {
        count = c;
    }
    void operator++() {
        ++count;
    }
    void operator++(int) {
        count++;
    }
    void display {
        cout << "Student count:" << count << endl;
    }
};

int main() {
    student s1(50);
    cout << "Before increment:" << endl;
    s1.display();

    ++s1;
    cout << "After pre-increment:" << endl;
    s1.display();
}
```

```

    s++;  
    cout << "After post-increment:" << endl;  
    s1.display();  
    return 0;  
}

```

Ans
24

Experiment-8

Q.1] WAP to overload the '+' operator so that 2 strings can be concatenated.

```

#include <iostream>
#include <string>
using namespace std;

```

```

class combine {
    string str;
public:
    combine (string s = " ") {
        str = s;
    }
    combine operator+(const combine &obj) {
        return combine (str+obj.str);
    }
    void display() {
        cout << str << endl;
    }
};

int main() {
    combine s1("xyz"), s2("pqr"), s3;
    s3 = s1 + s2;
    cout << "Concatenated string:";
    s3.display();
}

```


Q2) WAP to create a base class Ilogin having data members name and password. Declare accept() function virtual. Derive Email login and Membership login class from Ilogin. Display Email login details and membership login details of the employee.

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Ilogin {
protected:
    string name, password;
public:
    virtual void accept() {
        cout << "Enter name:";
        cin >> name;
        cout << "Enter password:";
        cin >> password;
    }
    virtual void display() {
        cout << "Name: " << name << endl;
        cout << "Password: " << password << endl;
    }
};
```

```
class Emaillogin : public Ilogin {
public:
    string email;
    void accept() override {
        cout << "Enter Email ID:";
```

```
        cin >> email;
        Ilogin::accept();
    }
    void display() override {
        cout << "In Email login details" << endl;
        cout << "Email ID:" << email << endl;
        Ilogin::display();
    }
};

class Membershiplogin : public Ilogin {
    string memberID;
public:
    void accept() override {
        cout << "Enter membership ID:";
        cin >> memberID;
        Ilogin::accept();
    }
    void display() override {
        cout << "In Membership login details" << endl;
        cout << "Membership ID:" << memberID << endl;
        Ilogin::display();
    }
};

int main() {
    Ilogin *login;
```

```
    Emaillogin e;
    Membershiplogin m;
```

login = & e;
login → accept ();
login → display ();

login & m;
login → accept ();
login → display ();

return 0;

}

~~Q~~
4/11

Experiment - 9.

- 1) WAP to copy the contents in one file into another. Open "First.txt" in read mod and "Second.txt". Assume "First.txt" is already created.

```
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main() {
    ifstream inFile ("First.txt");
    ofstream outFile ("Second.txt");
```

```
    if (!inFile) {
        cout << "Error opening First.txt" << endl;
        return 1;
    }
```

```
    char ch;
    while (inFile.get (ch)) {
        outFile.put (ch);
    }
```

```
    cout << "File copied successfully" << endl;
    inFile.close ();
    outFile.close ();
    return 0;
```


2) WAP to count digits and spaces using file handling.

```
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main() {
    ifstream file ("first.txt");
    if (!file) {
        cout << "Error opening file" << endl;
        return 1;
    }
```

```
    char ch;
    int digits = 0; spaces = 0;
```

```
    while (file.get(ch)) {
        if (is digit(ch))
            digits++;
        else if (is space(ch))
            spaces++;
    }
```

```
    cout << "Digits : " << digits << endl;
    cout << "Spaces : " << spaces << endl;
```

```
    file.close();
    return 0;
}
```

3) WAP to count words using file handling.

```
#include <iostream>
#include <fstream>
#include <string>
#include <string>
using namespace std;
```

```
int main() {
    ifstream file ("first.txt");
    if (!file) {
        cout << "Error" << endl;
        return 1;
    }
```

```
    string word;
    int count = 0;
```

```
    while (file >> word)
        count++;
```

```
    cout << "Total Words : " << count << endl;
```

```
    file.close();
    return 0;
}
```

4) WAP to count occurrence of a given word using File Handling.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
```

```
int main() {
    ifstream file("First.txt");
    if (!file) {
        cout << "Error " << endl;
        return 1;
    }
```

```
    string word, target;
    int count = 0;
```

```
    cout << "Enter word to count : " << endl;
    cin >> target;
```

```
    while (file >> word) {
        if (word == target)
            count ++;
```

```
    }
    cout << "Occurrence of " << target << " : "
        << count << endl;
```

```
    file.close();
```

```
}
```


Experiment 10

```
A) #include <iostream>
using namespace std;

template <class T>
T sum Array(T arr[], int n){
    T sum = 0;
    for(int i=0; i<n; i++)
        sum += arr[i];
    return sum;
}

int main(){
    int int_arr[5] = {1, 2, 3, 4, 5};
    float flo_arr[5] = {1.1, 2.2, 3.3, 4.4, 5.5};
    double dec_arr[5] = {0.5, 1.5, 2.5, 3.5, 4.5};

    cout << "Sum of integer array:" << sum Array(int_arr, 5)
        << endl;
    cout << "sum of Float array:" << sum Array(flo_arr, 5)
        << endl;
    cout << "sum of Double array:" << sum Array(dec_arr, 5)
        << endl;

    return 0;
}
```

```

B) #include <iostream>
#include <string>
using namespace std;

```

```

template <typename T>
T square(T num){
    return num * num;
}

```

```

template <>
string square<string>(string str){
    return str + str;
}

```

```

int main(){
    int num = 5;
    double dec-num = 2.5;
    string str = "Apple";

```

```

    cout << "square of integer:" << square(num);
    cout << "square of double:" << square(dec-num);
    cout << "square of string:" << square(str) << endl;

```

```

    return 0;
}

```

```

C) #include <iostream>
#include <math.h>
using namespace std;

```

```

template <class T1, class T2> class calc {
public:
    T1 x;
    T2 y;

```

```

    calc(T1 n, T2 m){
        x = n;
        y = m;
    }

```

```

    void sum(){
        cout << "Sum:" << x + y << endl;
    }

```

```

    void proc(){
        cout << "Product:" << x * y << endl;
    }

```

```

    void diff(){
        cout << "Difference:" << x - y << endl;
    }

```

```

    void quo(){
        cout << "Quotient:" << x / y << endl;
    }

```

```

    void rem(){
        cout << "Remainder:" << x % y << endl;
    }

```

```

    void power(){
        cout << "X raised to Y is:" << pow(x, y) << endl;
    }
}

```



```

void min-num() {
    cout << "min of number: " << fmin(x, y) << endl;
}

void max-num() {
    cout << "Max of numbers: " << fmax(x, y) << endl;
}

void sin-x() {
    cout << "sin of x: " << sin(x) << endl;
}

void cos-y() {
    cout << "cos of y: " << cos(y) << endl;
}

};

int main() {
    calc << int, int > n(100, 200);

    n.sum();
    n.diff();
    n.pow();
    n.quo();
    n.rem();
    n.power();
    n.min-num();
    n.max-num();
    n.sin-x();
    n.cos-y();
}

```

Pr
||||

Experiment - 11

```

A) #include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    cout << "Initial vector: " << endl;
    for (int i = 0; i < 10; i++) {
        cout << v[i] << " " << endl;
    }

    cout << "Multiply by 10" << endl;
    for (int i = 0; i < 10; i++) {
        v[i] = v[i] * 10;
    }

    cout << "New vector: " << endl;
    for (int i = 0; i < 10; i++) {
        cout << v[i] << " " << endl;
    }

    return 0;
}

```

B) [with iterators]

```
#include <iostream>
#include <vector>
using namespace std;
```

```
int main() {
```

```
vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

```
cout << "Initial vector:" << endl;
```

```
for (vector<int>::iterator it = v.begin();
     it != v.end(); ++it) {
```

```
    cout << endl;
```

```
    cout << "Multiply by 10" << endl;
```

```
    for (vector<int>::iterator it = v.begin(); it !=
         v.end(); ++it) {
        *it = (*it) * 10;
```

```
    }
```

```
    cout << "New vector:" << endl;
```

```
    for (vector<int>::iterator it = v.begin();
         it != v.end(); ++it) {
        *it = (*it) * 10;
```

```
    }
```

```
    cout << "New vector:" << endl;
```

```
    for (vector<int>::iterator it = v.begin();
         it != v.end(); ++it) {
        cout << *it << " ";
```

```
    } cout << endl;
```

```
    return 0;
```

```
}
```

[Handwritten signature]

Experiment - 12

Ques WAP to implement stack.

```
#include <iostream>
#include <stack>
using namespace std;
```

```
stack<int> stack 1;
```

```
void disp() {
```

```
    stack<int> temp = stack 1;
```

```
    while (!temp.empty()) {
```

```
        cout << temp.top() << " ";
```

```
        temp.pop();
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
int main() {
```

```
    int num;
```

```
    cout << "Enter a no:" << endl;
```

```
    cin >> num;
```

```
    for (int i = 0; i < num; i++) {
```

```
        int temp;
```

```
        cout << "Enter element at position" << i+1 << endl;
```

```
        cin >> temp;
```

```
        stack 1.push(temp);
```

```
    }
```

```
    cout << endl;
```

```
    cout << "Top most element:" << stack 1.top() << endl;
```

```
    cout << endl;
```

```
}
```



```
cout << "Stack elements (top to bottom)---" << endl;
disp();
cout << endl;
cout << "Pop Function:" << endl;
stack1.pop();
disp();
stack2.pop();
disp();
stack3.pop();
disp();
}
```

Que WAP to implement STL Queue.

```
#include <iostream>
#include <queue>
using namespace std;
```

```
int main() {
    queue <int> q;
    for (int i = 0; i < 10; i++) {
        q.push(i * 10);
    }
    cout << endl;
    cout << "Front element" << q.front() << endl;
    cout << "Back element" << q.back() << endl;
    cout << endl;
    q.pop();
    cout << "After one pop, front = " << q.front() << endl;
    cout << endl;
    cout << "Queue elements (front to back:)" << endl;
    while (!q.empty()) {
        cout << q.front() << " ";
    }
}
```

Pr
4/11